Especificación de Requerimientos

AbejaNet — Aplicación Móvil

Prototipo Versión 3

Materia:
Administración de Proyectos de TI

Carrera:
Ingeniería en Desarrollo y Gestión de Software

Grupo:
IDGS 16

Periodo:
Mayo — Agosto 2026

Profesora:
Angélica Garduño Bustamante

Equipo de Trabajo

Marco Antonio Gómez Olvera	Programador de la Aplicación Móvil
Israel Gómez Bonilla	Desarrollador de la Aplicación Web (Administrador)
Orlando Cabrera Rubio	Encargado de IoT
Sandra Zoé Cabrera Velázquez	Encargada de Documentación

Unidad Tecnológica de Querétaro (UTEQ)

Querétaro, 5 de junio de 2026

Índice

1. Introducción **2**
1.1. Contexto 2
1.2. Motivación de las Mejoras 2
1.3. Objetivo del Documento 2

2. Alcance **4**
2.1. Dentro del Alcance 4
2.2. Fuera del Alcance 4
2.3. Supuestos 4

3. Control de Cambios **6**

4. Requerimientos Funcionales **7**
4.1. RQF-03 — Anti-spam y validación de email 8
4.2. RQF-04 — Indicador de conexión y caché offline del dashboard 11
4.3. RQF-05 — Bitácora apícola con calendario 14

5. Requerimientos No Funcionales **17**
5.1. RNF-01 — Soporte bilingüe Español / Inglés 18
5.2. RNF-02 — Retroalimentación háptica y toasts no intrusivos 21
5.3. RNF-03 — Bloqueo automático por inactividad 24
5.4. RNF-04 — Compatibilidad multiplataforma 27
5.5. RNF-05 — Tiempo de respuesta en operaciones locales 30

6. Diagramas de Casos de Uso **33**
6.1. RQF-03 — Anti-spam y validación de email 33
6.2. RQF-04 — Indicador de conexión y caché offline 33
6.3. RQF-05 — Bitácora apícola con calendario 35
6.4. Priorización MoSCoW 35

1 Introducción

1.1 Contexto

AbejaNet es una plataforma de monitoreo IoT para la apicultura que integra sensores ESP32, un backend en Node.js/PostgreSQL desplegado en Render, una aplicación web de administración y una aplicación móvil construida con React Native y Expo. La aplicación móvil permite a los apicultores supervisar en tiempo real la temperatura, humedad, peso y sonido de sus colmenas, así como recibir alertas críticas mediante notificaciones push.

El proyecto es desarrollado por un equipo multidisciplinario: **Marco Antonio Gómez Olvera** (programador de la aplicación móvil), **Israel Gómez Bonilla** (desarrollador de la aplicación web de administración), **Orlando Cabrera Rubio** (encargado de IoT) y **Sandra Zoé Cabrera Velázquez** (encargada de documentación).

Actualmente la aplicación cuenta con funcionalidades base implementadas: autenticación con JWT y refresh tokens, autenticación biométrica al reabrir la app, soporte de tema claro/oscuro, registro de usuarios con validación de contraseña fuerte, recuperación de contraseña por código de verificación, dashboard con resumen de apiarios, gestión de colmenas, alertas históricas y exportación de datos a CSV.

1.2 Motivación de las Mejoras

A partir de un análisis del código fuente y de la experiencia de uso, se identificaron áreas concretas en las que la aplicación puede elevar su calidad sin comprometer la estabilidad existente. Estas mejoras se concentran en tres ejes:

- **Experiencia de usuario (UX):** retroalimentación inmediata en formularios, internacionalización completa, retroalimentación háptica y notificaciones tipo *toast* no intrusivas.
- **Seguridad:** prevención de envíos múltiples de peticiones (anti-spam), bloqueo automático por inactividad y resiliencia ante caídas de conexión.
- **Plus funcional:** una bitácora apícola digital con calendario que sustituye los cuadernos físicos tradicionales del apicultor.

1.3 Objetivo del Documento

El presente documento especifica **únicamente los requerimientos de la aplicación móvil** de AbejaNet correspondientes al **Prototipo Versión 3**. Quedan fuera de su alcance los requerimientos de la aplicación web de administración, del backend (salvo el

soporte mínimo necesario para una funcionalidad móvil) y del subsistema IoT, los cuales se documentan por separado.

El documento se organiza en dos bloques: **requerimientos funcionales** y **requerimientos no funcionales**. Cada requerimiento es alcanzable dentro del cuatrimestre Mayo — Agosto 2026 y los requerimientos funcionales se complementan con un diagrama de casos de uso al final del documento.

2 Alcance

2.1 Dentro del Alcance

- Implementación en la aplicación móvil React Native (Expo SDK 51) del proyecto AbejaNet.
- Modificaciones puntuales al backend (Node.js / Express / PostgreSQL) únicamente para dar soporte a la bitácora apícola (RQF-05), funcionalidad propia de la app móvil.
- Internacionalización completa de las cadenas de texto que aún se encuentran codificadas en español dentro del frontend.
- Integración de las librerías `react-native-toast-message`, `@react-native-community/netinfo` y `react-native-calendars`.
- Reutilización del sistema de autenticación biométrica ya existente para el nuevo bloqueo por inactividad.

2.2 Fuera del Alcance

- Rediseño visual integral de pantallas ya existentes.
- Cambios de modelo de negocio o de roles de usuario.
- Integración con servicios externos de pago, mapas o análisis avanzado mediante inteligencia artificial.
- Modificaciones al firmware del ESP32 ni al protocolo de envío de lecturas.
- Requerimientos de la aplicación web de administración, del subsistema IoT (ESP32) o del backend que no estén directamente ligados a una funcionalidad de la app móvil; éstos se documentan por separado.
- Soporte para dispositivos wearables o Smart TV.

2.3 Supuestos

- El backend continuará desplegado en Render con la misma URL de producción.
- Los dispositivos de prueba contarán con Android 8.0 (API 26) o superior, o iOS 13.0 o superior.

- Los usuarios cuentan con conexión intermitente a internet; la app debe degradar correctamente cuando esta falle.
- Los apicultores que usen la bitácora tienen al menos una colmena registrada en el sistema.

3 Control de Cambios

Edición	Fecha de Última Edición
3	04/06/2026

Versión	Descripción del Cambio	Autor de Cambio	Rol	Fecha
1.0.0	Creación del documento de requerimientos de la aplicación móvil (Prototipo v3).	Marco Antonio Gómez Olvera	Autor / Programador App Móvil	01/06/2026
1.0.1	Revisión y estandarización documental del formato.	Sandra Zoé Cabrera Velázquez	Revisora / Encargada de Documentación	03/06/2026
1.0.2	Aprobación final del documento para entrega.	Angélica Garduño Bustamante	Docente / Aprobadora	04/06/2026

Este documento especifica los ocho requerimientos de la aplicación móvil AbejaNet correspondientes al Prototipo Versión 3, agrupados en **requerimientos funcionales** (tres) y **requerimientos no funcionales** (cinco). Cada ficha contiene la información del cliente, la información general del requerimiento, su descripción detallada y las condiciones de prueba.

4 Requerimientos Funcionales

Los requerimientos funcionales describen las capacidades y comportamientos concretos que la aplicación móvil debe ofrecer al usuario. La numeración continúa a partir de los requerimientos RQF-01 y RQF-02, ya implementados en prototipos anteriores (requerimiento general móvil y registro de clientes, respectivamente).

4.1 RQF-03 — Anti-spam y validación de email

1. INFORMACIÓN DEL CLIENTE	
Nombre de la empresa	UTEQ
Nombre del Contacto	Angélica Garduño Bustamante
Fecha de Elaboración	01/06/2026

2. INFORMACIÓN GENERAL	
Nombre del Requerimiento	RQF 03. Anti-spam y validación de email
Tipo de Requerimiento	Funcional
Fecha requerida de solución	30/06/2026
Prioridad	Alta

3. DESCRIPCIÓN DEL REQUERIMIENTO	
Objetivo del Requerimiento	Implementar un mecanismo de validación de email mediante expresión regular y un sistema anti-spam que deshabilite los botones de los formularios durante las peticiones HTTP, evitando envíos duplicados al backend.
Beneficio esperado	Reducción de la carga al servidor por peticiones duplicadas, mejora de la experiencia de captura de datos del usuario y prevención de errores de validación previos al envío.
Alcance	- Aplicar validación de email en *login*, *register* y *forgot-password*. - Deshabilitar el botón de envío durante la petición HTTP. - Mostrar borde rojo y mensaje inline en inputs inválidos. - No modifica la lógica del backend.
Ubicación	https://github.com/MarkG777/AbejaNet
Descripción de Usuarios y Responsables	**Usuarios:** **Usuario (Cliente):** interactúa con los formularios de autenticación de la app móvil. **Responsables:** **App Móvil (React Native):** ejecuta la validación local antes de enviar la petición. **API Backend:** no requiere cambios; solo recibe peticiones bien formadas.
Descripción funcional	1. El usuario ingresa al formulario de inicio de sesión, registro o recuperación de contraseña. 2. Al escribir el email, el sistema valida el formato en tiempo real con una expresión regular. 3. Si el formato es inválido, el input muestra borde rojo y mensaje inline; el botón permanece deshabilitado. 4. Cuando los datos son válidos y el usuario presiona el botón de envío, el botón se deshabilita inmediatamente y muestra un indicador de carga. 5. Mientras la petición está en curso, el formulario no acepta nuevas interacciones. 6. Al recibirse la respuesta del backend, el botón se reactiva y se muestra el resultado correspondiente.
Restricciones	- La validación local no sustituye la validación del backend. - Debe mantenerse la compatibilidad con el sistema de autenticación JWT existente.

4. CONDICIONES DE PRUEBA		
Condición	**Validación**	**Resultado**
El usuario ingresa un email con formato inválido en la pantalla de inicio de sesión.	El sistema debe detectar el formato incorrecto antes de enviar la petición y mostrar un mensaje inline.	El input se marca con borde rojo y el botón de envío permanece deshabilitado.
El usuario presiona repetidamente el botón *"Iniciar sesión"* mientras la petición está en curso.	El sistema debe deshabilitar el botón al primer clic para evitar envíos duplicados.	Únicamente se realiza una petición HTTP al backend, sin importar el número de clics.

4.2 RQF-04 — Indicador de conexión y caché offline del dashboard

1. INFORMACIÓN DEL CLIENTE	
Nombre de la empresa	UTEQ
Nombre del Contacto	Angélica Garduño Bustamante
Fecha de Elaboración	01/06/2026

2. INFORMACIÓN GENERAL	
Nombre del Requerimiento	RQF 04. Indicador de conexión y caché offline del dashboard
Tipo de Requerimiento	Funcional
Fecha requerida de solución	15/07/2026
Prioridad	Alta

3. DESCRIPCIÓN DEL REQUERIMIENTO	
Objetivo del Requerimiento	Implementar un sistema de detección de conectividad y caché local del dashboard que permita al usuario consultar los últimos datos disponibles cuando no exista conexión a internet, mostrando un indicador visual claro de su estado.
Beneficio esperado	La aplicación se mantiene utilizable durante caídas de conexión o *cold starts* del backend. El usuario obtiene información reciente sin pantallas de carga infinita y comprende el estado de la red en todo momento.
Alcance	- Integrar la librería `@react-native-community/netinfo`. - Mostrar un banner superior cuando no haya conexión. - Cachear el último resumen del dashboard en `AsyncStorage`. - Refrescar automáticamente al recuperar conexión.
Ubicación	https://github.com/MarkG777/AbejaNet
Descripción de Usuarios y Responsables	**Usuarios:** **Usuario (Cliente):** consulta el dashboard con o sin conexión a internet. **Responsables:** **App Móvil:** detecta el estado de la red y administra el caché. **AsyncStorage:** persiste localmente el último resumen recibido. **API Backend:** provee los datos cuando hay conexión disponible.
Descripción funcional	1. Al abrir el dashboard, la app consulta el estado de la red mediante `NetInfo`. 2. Si hay conexión, se descargan los datos del backend y se almacenan en caché con marca de tiempo. 3. Si no hay conexión, se muestra un banner superior amarillo con el mensaje *"Sin conexión"*. 4. La pantalla muestra los últimos datos cacheados con la leyenda *"Actualizado hace X min"*. 5. Cuando la conexión se restablece, la app refresca automáticamente los datos. 6. El banner desaparece al confirmarse la reconexión.
Restricciones	- El caché debe limitarse al último resumen del dashboard para evitar consumo excesivo de almacenamiento. - La marca de tiempo debe persistirse junto con los datos.
Precedencias	RQF 01. Requerimiento general Móvil

4. CONDICIONES DE PRUEBA		
Condición	**Validación**	**Resultado**
El usuario activa el modo avión con la app abierta en el dashboard.	La app debe detectar la pérdida de conexión y reaccionar visualmente.	Aparece el banner *"Sin conexión"* y se muestran los datos cacheados con marca de tiempo.
El usuario abre la app sin conexión por primera vez tras tener datos cacheados.	La app debe cargar los datos desde `AsyncStorage` sin intentar petición HTTP.	El dashboard se renderiza con los datos cacheados en menos de 2 segundos.

4.3 RQF-05 — Bitácora apícola con calendario

1. INFORMACIÓN DEL CLIENTE	
Nombre de la empresa	UTEQ
Nombre del Contacto	Angélica Garduño Bustamante
Fecha de Elaboración	01/06/2026

2. INFORMACIÓN GENERAL	
Nombre del Requerimiento	RQF 05. Bitácora apícola con calendario
Tipo de Requerimiento	Funcional
Fecha requerida de solución	30/07/2026
Prioridad	Alta

3. DESCRIPCIÓN DEL REQUERIMIENTO	
Objetivo del Requerimiento	Implementar una bitácora apícola digital con calendario interactivo que permita al apicultor registrar, consultar, editar y eliminar eventos asociados a sus colmenas y apiarios, sustituyendo la libreta de campo tradicional.
Beneficio esperado	Centralización del historial de inspecciones, tratamientos, cosechas y cambios de reina en un solo lugar accesible desde el dispositivo móvil. Mejora la trazabilidad de las labores apícolas y facilita la toma de decisiones informadas.
Alcance	- Crear nueva pantalla *Bitácora* accesible desde el *drawer* del usuario. - Mostrar un calendario mensual con dots de color en los días con eventos. - CRUD completo de eventos clasificados por tipo. - Acceso rápido desde la pantalla de detalle de cada colmena. - Crear tabla `bitacora` en PostgreSQL y endpoints REST asociados.
Ubicación	https://github.com/MarkG777/AbejaNet
Descripción de Usuarios y Responsables	**Usuarios:** **Apicultor (Cliente):** registra y consulta eventos sobre el manejo de sus colmenas. **Responsables:** **App Móvil:** ofrece la interfaz de calendario y los formularios de evento. **API Backend:** expone los endpoints `/api/bitacora` para CRUD. **Base de Datos PostgreSQL:** almacena los eventos en la tabla `bitacora` con relaciones a `usuarios`, `colmenas` y `apiarios`.
Descripción funcional	1. El apicultor accede a *Bitácora* desde el *drawer* del usuario. 2. Visualiza un calendario mensual con dots de color en los días con eventos registrados. 3. Selecciona un día y consulta el listado de eventos de esa fecha. 4. Mediante un botón flotante (FAB) crea un nuevo evento eligiendo tipo (Inspección, Cosecha, Tratamiento, Cambio de reina, División/Unión, Alimentación o Nota libre), colmena/apiario asociado, título y comentario. 5. Puede editar o eliminar eventos existentes.

4. CONDICIONES DE PRUEBA		
Condición	**Validación**	**Resultado**
El apicultor crea un nuevo evento de tipo *Inspección* desde la pantalla de bitácora.	El sistema debe persistir el evento en el backend y reflejarlo en el calendario.	El día correspondiente muestra un dot de color y al seleccionarlo aparece el evento creado.
El apicultor accede a *"Registrar evento"* desde el detalle de una colmena.	El formulario debe abrirse con la colmena preseleccionada en el campo correspondiente.	El evento se guarda asociado correctamente a esa colmena sin requerir selección manual.

5 Requerimientos No Funcionales

Los requerimientos no funcionales describen las cualidades transversales de la aplicación móvil (usabilidad, seguridad, portabilidad y rendimiento). Al no representar interacciones discretas del usuario, no se acompañan de diagramas de casos de uso.

5.1 RNF-01 — Soporte bilingüe Español / Inglés

1. INFORMACIÓN DEL CLIENTE	
Nombre de la empresa	UTEQ
Nombre del Contacto	Angélica Garduño Bustamante
Fecha de Elaboración	01/06/2026

2. INFORMACIÓN GENERAL	
Nombre del Requerimiento	RNF 01. Soporte bilingüe Español / Inglés
Tipo de Requerimiento	No funcional — Usabilidad
Fecha requerida de solución	25/06/2026
Prioridad	Media

3. DESCRIPCIÓN DEL REQUERIMIENTO	
Objetivo del Requerimiento	Garantizar que la totalidad de las cadenas de texto visibles para el usuario estén disponibles en español e inglés, permitiendo el cambio dinámico del idioma desde la pantalla de perfil.
Beneficio esperado	Accesibilidad para usuarios anglófonos, experiencia profesional consistente y posibilidad de uso académico internacional del proyecto.
Alcance	- Extraer las cadenas hardcoded restantes (`login`, `register`, `onboarding`, alertas) hacia los archivos `locales/es.json` y `locales/en.json`. - Localizar el formato de fechas según el idioma activo. - Persistir la preferencia de idioma en `AsyncStorage`.
Ubicación	https://github.com/MarkG777/AbejaNet
Descripción de Usuarios y Responsables	**Usuarios:** **Usuario (Cliente):** cambia el idioma desde el perfil. **Responsables:** **App Móvil:** aplica el idioma seleccionado a través de `react-i18next`. **AsyncStorage:** persiste la preferencia entre sesiones.
Descripción funcional	1. El usuario accede a su perfil y elige el idioma deseado. 2. Toda la interfaz se actualiza al instante sin reiniciar la app. 3. La preferencia se almacena localmente y se aplica al siguiente arranque. 4. Los formatos de fecha y hora se muestran según la convención del idioma activo.
Restricciones	- Solo se soportan español e inglés en esta iteración. - Las claves nuevas deben mantener consistencia entre ambos archivos JSON.
Precedencias	Ninguna (la base de `i18next` ya existe en el proyecto).
Documentación requerida	Manual técnico con la convención de claves de internacionalización.
Observaciones	La librería `react-i18next` ya se encuentra instalada en el proyecto.

4. CONDICIONES DE PRUEBA		
Condición	**Validación**	**Resultado**
El usuario cambia el idioma de español a inglés desde el perfil.	La interfaz debe traducirse en tiempo real, incluyendo formularios, validaciones y alertas.	Todos los textos visibles cambian al inglés sin reiniciar la app.
El usuario cierra y vuelve a abrir la app después de haber cambiado el idioma.	La preferencia debe persistir entre sesiones.	La app inicia automáticamente con el idioma previamente seleccionado.

5.2 RNF-02 — Retroalimentación háptica y toasts no intrusivos

1. INFORMACIÓN DEL CLIENTE	
Nombre de la empresa	UTEQ
Nombre del Contacto	Angélica Garduño Bustamante
Fecha de Elaboración	01/06/2026

2. INFORMACIÓN GENERAL	
Nombre del Requerimiento	RNF 02. Retroalimentación háptica y toasts no intrusivos
Tipo de Requerimiento	No funcional — Usabilidad
Fecha requerida de solución	30/06/2026
Prioridad	Media

3. DESCRIPCIÓN DEL REQUERIMIENTO	
Objetivo del Requerimiento	Sustituir las alertas modales (`Alert.alert`) por notificaciones tipo *toast* no intrusivas y agregar retroalimentación háptica en las acciones críticas de la app.
Beneficio esperado	La aplicación adquiere una sensación nativa y fluida, alineada con las mejores prácticas de apps móviles modernas. El usuario recibe confirmación de sus acciones sin interrupciones de flujo.
Alcance	- Integrar la librería `react-native-toast-message`. - Aprovechar `expo-haptics` (ya instalada) en login, guardado de perfil, registro de evento y marcado de alertas como leídas. - Mantener `Alert.alert` solo para confirmaciones destructivas (eliminar cuenta, eliminar registros).
Ubicación	https://github.com/MarkG777/AbejaNet
Descripción de Usuarios y Responsables	**Usuarios:** **Usuario (Cliente):** ejecuta acciones que generan retroalimentación háptica y visual. **Responsables:** **App Móvil:** dispara la vibración y muestra los toasts en respuesta a las acciones del usuario.
Descripción funcional	1. Cuando el usuario ejecuta una acción crítica, la app dispara una vibración suave mediante `expo-haptics`. 2. El resultado de la acción se muestra como toast (verde para éxito, rojo para error, amarillo para advertencia) en la parte inferior de la pantalla. 3. El toast desaparece automáticamente tras 3 segundos sin bloquear la interfaz. 4. Las acciones destructivas continúan mostrando confirmación modal previa.
Restricciones	- La intensidad háptica debe ser `Light` para no resultar molesta. - Los toasts no deben superponerse con la barra de navegación inferior.
Precedencias	Ninguna (las dependencias ya están en el proyecto).
Documentación requerida	No se requiere documentación adicional para el usuario final.
Observaciones	La librería `expo-haptics` ya es utilizada parcialmente en el

4. CONDICIONES DE PRUEBA		
Condición	**Validación**	**Resultado**
El usuario guarda cambios en su perfil correctamente.	La app debe vibrar suavemente y mostrar un toast de éxito.	Aparece un toast verde *"Perfil actualizado"* y se siente la vibración háptica.
El usuario ingresa credenciales incorrectas en el inicio de sesión.	La app debe mostrar un toast rojo en lugar de un `Alert.alert`.	Aparece un toast rojo *"Credenciales inválidas"* sin bloquear la pantalla de login.

5.3 RNF-03 — Bloqueo automático por inactividad

1. INFORMACIÓN DEL CLIENTE	
Nombre de la empresa	UTEQ
Nombre del Contacto	Angélica Garduño Bustamante
Fecha de Elaboración	01/06/2026

2. INFORMACIÓN GENERAL	
Nombre del Requerimiento	RNF 03. Bloqueo automático por inactividad
Tipo de Requerimiento	No funcional — Seguridad
Fecha requerida de solución	20/07/2026
Prioridad	Alta

3. DESCRIPCIÓN DEL REQUERIMIENTO	
Objetivo del Requerimiento	Bloquear automáticamente la aplicación tras un periodo de inactividad del usuario en primer plano, exigiendo autenticación biométrica para reanudar el uso y proteger la información sensible del apiario.
Beneficio esperado	Capa adicional de seguridad sin requerir el reingreso constante de la contraseña. Protección de datos en escenarios donde el usuario deja el dispositivo desbloqueado por error.
Alcance	- Implementar un temporizador global de inactividad con duración predeterminada de 5 minutos. - Detectar interacciones del usuario mediante un *wrapper* a nivel de aplicación. - Reutilizar el sistema de `LocalAuthentication` ya existente para el bloqueo. - Cerrar sesión tras tres intentos biométricos fallidos.
Ubicación	https://github.com/MarkG777/AbejaNet
Descripción de Usuarios y Responsables	**Usuarios:** **Usuario (Cliente):** reanuda el uso de la app tras un periodo de inactividad. **Responsables:** **App Móvil:** mide el tiempo de inactividad y dispara la pantalla biométrica. **Sistema operativo:** provee la API de autenticación biométrica nativa.
Descripción funcional	1. Mientras la app está en primer plano, un temporizador global mide el tiempo desde la última interacción del usuario. 2. Si transcurren 5 minutos sin interacción, se dispara la pantalla biométrica. 3. Si la autenticación es exitosa, el usuario regresa a la pantalla en la que estaba. 4. Si falla 3 veces, la sesión se cierra y se redirige a la pantalla de inicio de sesión. 5. Cualquier interacción (toque, scroll) reinicia el temporizador.
Restricciones	- El temporizador se pausa cuando la app pasa a segundo plano (ya está cubierto por la biometría existente). - El umbral de 5 minutos no es configurable por el usuario en esta iteración.

4. CONDICIONES DE PRUEBA		
Condición	**Validación**	**Resultado**
El usuario deja la app abierta sin interacción por más de 5 minutos.	El sistema debe disparar la pantalla biométrica de bloqueo.	Aparece la solicitud de huella o Face ID; al validarse, el usuario regresa a la pantalla previa.
El usuario falla tres veces consecutivas la autenticación biométrica de bloqueo.	El sistema debe cerrar la sesión por seguridad.	La app cierra sesión y redirige al usuario a la pantalla de inicio de sesión.

5.4 RNF-04 — Compatibilidad multiplataforma

1. INFORMACIÓN DEL CLIENTE	
Nombre de la empresa	UTEQ
Nombre del Contacto	Angélica Garduño Bustamante
Fecha de Elaboración	01/06/2026

2. INFORMACIÓN GENERAL	
Nombre del Requerimiento	RNF 04. Compatibilidad multiplataforma
Tipo de Requerimiento	No funcional — Portabilidad
Fecha requerida de solución	30/07/2026
Prioridad	Media

3. DESCRIPCIÓN DEL REQUERIMIENTO	
Objetivo del Requerimiento	Garantizar que la aplicación móvil ejecute correctamente todas sus funcionalidades, incluidas las nuevas, en Android 8.0 (API 26) o superior y en iOS 13.0 o superior.
Beneficio esperado	Cobertura de la mayoría de los dispositivos en uso por el público objetivo (apicultores) sin necesidad de hardware reciente. Reduce el riesgo de exclusión por incompatibilidad.
Alcance	- Declarar las versiones mínimas en `app.json`. - Validar las nuevas dependencias contra las versiones mínimas. - Probar en al menos un emulador de Android 8 y un simulador de iOS 13.
Ubicación	https://github.com/MarkG777/AbejaNet
Descripción de Usuarios y Responsables	**Usuarios:** **Usuario (Cliente):** ejecuta la app desde un dispositivo móvil de gama media. **Responsables:** **Equipo de desarrollo:** configura las versiones mínimas y ejecuta pruebas en emuladores. **Expo SDK 51:** provee la base de compatibilidad multiplataforma.
Descripción funcional	1. La app debe instalarse y ejecutarse en Android 8.0 o superior. 2. La app debe instalarse y ejecutarse en iOS 13.0 o superior. 3. Todas las funcionalidades nuevas (RQF 03 a RQF 05) deben verificarse en ambos sistemas operativos. 4. Las versiones mínimas se documentan en `app.json` y en el `README`.
Restricciones	- No se garantiza compatibilidad con Android < 8.0 ni iOS $<$ 13.0. - Las funciones biométricas dependen del hardware del dispositivo.
Precedencias	Ninguna.
Documentación requerida	`README` actualizado con las versiones mínimas.
Observaciones	Esta compatibilidad ya se garantiza por defecto al utilizar Expo SDK 51.

4. CONDICIONES DE PRUEBA		
Condición	**Validación**	**Resultado**
La app se instala en un emulador de Android 8.0 (API 26).	Todas las pantallas y funcionalidades deben renderizarse sin errores.	La app inicia correctamente y permite navegar por todas las pantallas.
La app se instala en un simulador de iOS 13.0.	La biometría, las notificaciones y el almacenamiento seguro deben funcionar.	Todas las funciones críticas operan sin advertencias de compatibilidad.

5.5 RNF-05 — Tiempo de respuesta en operaciones locales

1. INFORMACIÓN DEL CLIENTE	
Nombre de la empresa	UTEQ
Nombre del Contacto	Angélica Garduño Bustamante
Fecha de Elaboración	01/06/2026

2. INFORMACIÓN GENERAL	
Nombre del Requerimiento	RNF 05. Tiempo de respuesta en operaciones locales
Tipo de Requerimiento	No funcional — Rendimiento
Fecha requerida de solución	30/07/2026
Prioridad	Media

3. DESCRIPCIÓN DEL REQUERIMIENTO	
Objetivo del Requerimiento	Garantizar que las operaciones locales de la aplicación se completen en un tiempo máximo de dos (2) segundos en dispositivos de gama media, sin bloquear el hilo principal de la interfaz.
Beneficio esperado	Sensación de fluidez y respuesta inmediata en las acciones del usuario, alineada con los estándares de apps móviles modernas.
Alcance	- Cambio de pantalla mediante *Expo Router*. - Cambio de idioma desde el perfil. - Apertura de la pantalla de bitácora. - Validación de formularios. - Lectura del caché del dashboard.
Ubicación	`https://github.com/MarkG777/AbejaNet`
Descripción de Usuarios y Responsables	**Usuarios:** **Usuario (Cliente):** percibe la fluidez de la aplicación. **Responsables:** **Equipo de desarrollo:** mide y documenta los tiempos de respuesta usando `performance.now()` en dispositivos de prueba.
Descripción funcional	1. Las operaciones locales no deben exceder los 2 segundos en su ejecución. 2. Las operaciones que dependan de la red están exentas de este requerimiento. 3. Las animaciones deben ejecutarse a 60 FPS sin caídas perceptibles. 4. Los tiempos se documentan en una tabla del `README` técnico.
Restricciones	- La métrica aplica solo a operaciones locales (sin red). - Se considera *gama media* a un dispositivo con al menos 4 GB de RAM.
Precedencias	Ninguna.
Documentación requerida	Tabla de tiempos medidos en el `README` técnico.
Observaciones	Las dependencias actuales (`react-native-reanimated`) ya facilitan el cumplimiento del requerimiento.

4. CONDICIONES DE PRUEBA		
Condición	**Validación**	**Resultado**
El usuario navega del dashboard a la pantalla de bitácora.	La transición debe completarse en menos de 2 segundos.	La pantalla de bitácora se renderiza con el calendario visible dentro del umbral establecido.
El usuario cambia el idioma desde el perfil.	La interfaz debe actualizarse en tiempo real sin bloquear la UI.	La traducción se aplica en menos de 1 segundo y la app responde inmediatamente a nuevas interacciones.

6 Diagramas de Casos de Uso

A continuación se presentan los diagramas de casos de uso correspondientes a los tres requerimientos funcionales (RQF-03, RQF-04 y RQF-05) descritos en la sección de Requerimientos Funcionales. Los requerimientos no funcionales no se diagraman por tratarse de cualidades transversales del sistema y no de interacciones discretas del usuario.

6.1 RQF-03 — Anti-spam y validación de email

El siguiente diagrama describe la interacción del usuario con cualquiera de los formularios de autenticación (inicio de sesión, registro o recuperación de contraseña). La validación local previene el envío de peticiones HTTP con datos malformados y el bloqueo del botón impide envíos duplicados.

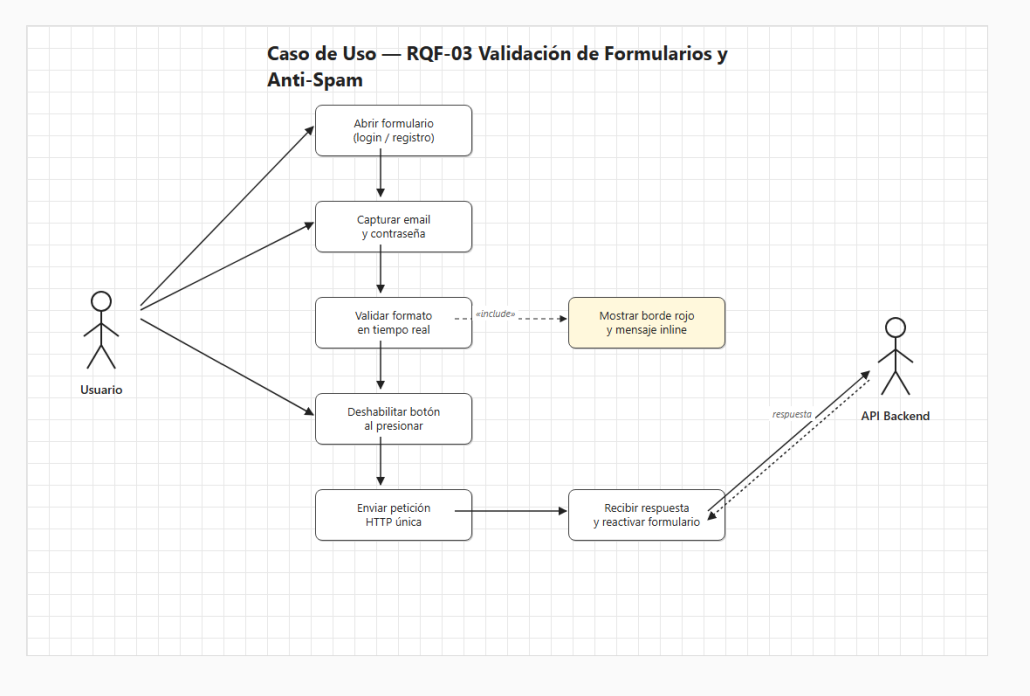


Figura 1: Diagrama de casos de uso — Validación de formularios y anti-spam

6.2 RQF-04 — Indicador de conexión y caché offline

El siguiente diagrama describe el comportamiento del dashboard al detectarse un cambio en la conectividad del dispositivo. El sistema decide entre mostrar datos en vivo, datos cacheados o un banner informativo según el estado de la red.

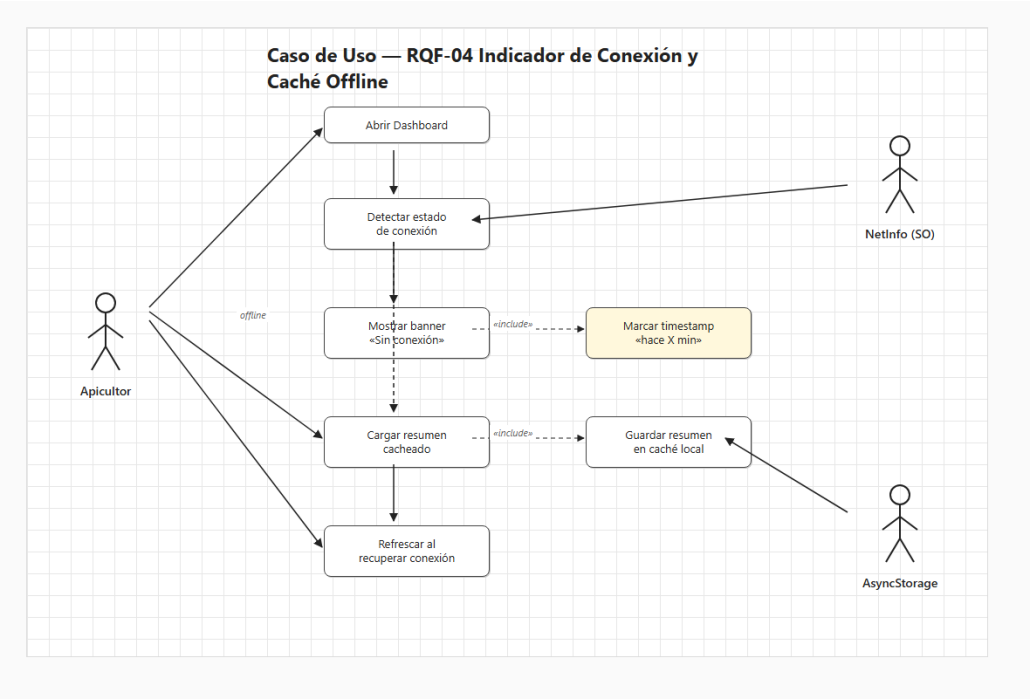


Figura 2: Diagrama de casos de uso — Indicador de conexión y caché offline

6.3 RQF-05 — Bitácora apícola con calendario

El siguiente diagrama describe las interacciones del apicultor con la nueva pantalla de bitácora y con el acceso rápido desde la pantalla de detalle de colmena. Ambos puntos de entrada culminan en la persistencia del evento en el backend.

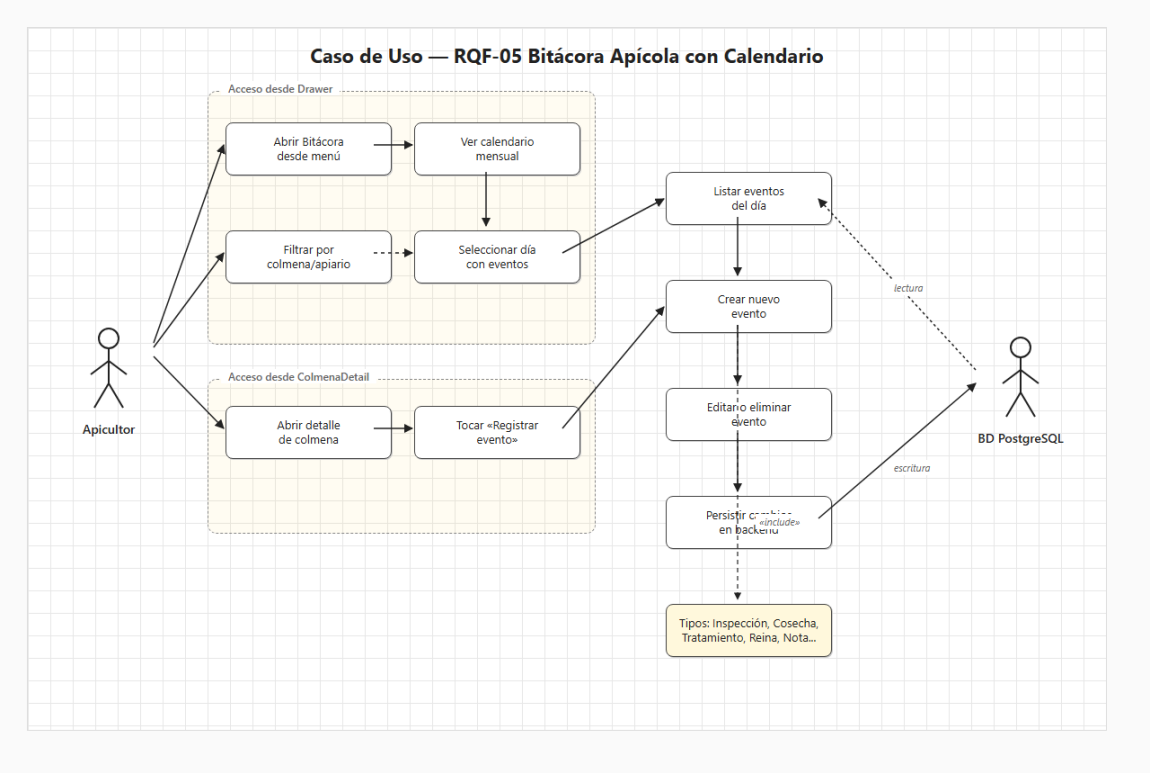


Figura 3: Diagrama de casos de uso — Bitácora apícola con calendario

6.4 Priorización MoSCoW

Tabla 1: Priorización de los nuevos requerimientos

Prioridad	**Requerimientos**
Must have	RQF-03 (anti-spam y validación), RQF-04 (caché offline), RNF-03 (bloqueo por inactividad)
Should have	RQF-05 (bitácora apícola), RNF-01 (soporte bilingüe), RNF-02 (háptico y toasts)
Could have	RNF-04 (compatibilidad multiplataforma validada), RNF-05 (rendimiento $<$ 2s medido y documentado)
Won't have	Rediseño visual integral, soporte para wearables/Smart TV e integración con pagos, mapas o IA (fuera del alcance de esta iteración)